



机器人导论课程大作业：基于学习的机器人控制

在作业四中，我们基于 DH 参数和逆运动学实现了机械臂的 Pick-and-Place 控制。这种方法虽然精确可靠，但需要手工推导运动学模型、设计状态机和轨迹规划，在面对环境变化或任务调整时需要大量人工修改。

本次作业将在作业四的基础上，让同学们自行选择一种 **基于学习的方法** 实现对机器人的控制，可以使用模仿学习、强化学习方法等。

1. 任务说明

完成与作业四相同的 Pick-and-Place 任务，可选方向包括但不限于：

模仿学习 (Imitation Learning)：以作业四的 IK 控制器为专家，收集其演示数据，训练神经网络模仿专家策略。

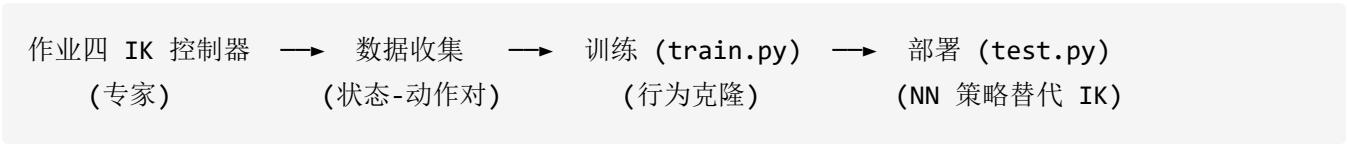
强化学习 (Reinforcement Learning)：不依赖专家演示，通过与仿真环境的交互和奖励信号自主学习控制策略。实现难度较高，鼓励大家探索并使用强化学习方法，视实验结果额外加分。

2. 基于学习的方法

2.1 模仿学习

行为克隆是模仿学习中最基础的方法，核心思想是：给定一组专家（hw4 中的 IK 控制器）的状态-动作对，通过监督学习训练一个神经网络，使其学会从观测到动作的映射。训练完成后，神经网络可以直接替代专家策略，完成抓取-放置任务。

作业四中完成的 IK 控制器可以作为本作业的**专家策略 (Expert Policy)** ——我们用它来生成训练数据，然后训练一个神经网络来模仿它，网络模型可以自行选择。



同学们可以根据所选方法自行设计网络结构和策略模型，选择恰当的观测变量。比如，可以采用如下的观测空间定义：

观测	维度	说明
方块位置	3	红色方块的世界坐标 (x, y, z)
目标位置	3	绿色标记的世界坐标 (x, y, z)
臂关节位置	4	joint1, joint2, joint4, joint6 角度 (rad)
臂关节速度	4	对应角速度 (rad/s)
末端执行器位置	3	hand 的世界坐标 (x, y, z)
上一时刻采取的动作	5	四个关节角度与一个夹爪开合度

2.2 强化学习

强化学习不依赖专家，而是让智能体在与环境的反复交互中，通过最大化累积奖励，自主学习策略。与行为克隆相比，强化学习无需专家数据，但对 **奖励函数设计** 和 **训练稳定性** 要求更高。

算法方面，适用于连续动作空间控制任务的方法包括 DDPG、TD3、SAC、PPO 等。同学可根据自身熟悉程度自行选择算法实现，其中 TD3 和 SAC 通常在机械臂连续控制任务中具有较好的训练稳定性和性能表现。

对于 Pick-and-Place 任务，**奖励函数的设计** 是核心挑战。
需要注意的是，本作业中的 Pick-and-Place 任务属于长时序、多阶段操作任务，需要依次完成接近目标、抓取、搬运和放置等多个步骤，而最终任务成功信号相对稀疏。若奖励函数设计不合理，智能体可能难以获得有效学习信号，从而出现训练收敛缓慢、成功率较低等问题。因此，强化学习路线通常需要投入更多时间进行奖励函数设计、参数调优和训练过程分析。

3. 机械臂模型、场景说明、仿真环境接口

与作业四相同

4. 项目结构

```
franka/
├── env/                                # 仿真环境包（不可修改）
│   ├── __init__.py                    # init
│   ├── franka_env.py                  # FrankaEnv
│   └── tensor_franka_env.py           # TensorFrankaEnv
│
├── controllers/                        # Controller（需完成 nn_controller.py）
│   ├── __init__.py                    # init
│   ├── base_controller.py              # Base Controller
│   ├── ik_controller.py                # IK Controller（作业四内容）
│   └── nn_controller.py                # NNController（大作业内容）
│
├── franka_emika_panda/                 # MuJoCo 模型资产（不可修改）
│   ├── franka_pick.xml                 # 主场景
│   ├── panda.xml                       # Panda 臂模型定义（关节/执行器/惯性/网格）
│   └── assets/                          # STL/OBJ 网格文件
│
├── data/                               # 收集脚本采集的专家轨迹数据集
│   └── rand/                            # 示例：随机方块位置的数据
│       ├── episode_0000.npz
│       ├── episode_0001.npz
│       └── metadata.npz
│
├── outputs/                            # 视频输出目录（训练好的模型可以放在此处）
├── preview.py                          # 浏览场景
├── test.py                             # 对控制器进行评分（不可修改）
├── record.py                           # 将运行结果导出为视频，存放在 outputs 文件夹下
├── collect.py                           # 数据收集
└── train.py                            # 训练脚本
```

5. 环境配置

```
# 创建虚拟环境
conda create -n franka python=3.8
conda activate franka

# 安装依赖
pip install mujoco numpy

# 安装 PyTorch
# 请根据自身硬件环境和 CUDA 版本使用 conda 自行选择合适的 PyTorch 版本
```

6. 评分标准

评分主要依据所采用学习方法的实现质量、实验结果以及报告完成情况进行综合评定：

评分项	分值	要求
实验部分	60%	实现学习方法，完成训练与部署，并在固定点和随机点场景下进行评测
实验报告	40%	结构完整、论述清晰、图表规范，包含方法、实验结果与分析

7. 提交内容

需要提交以下材料：

- 1. 实验报告（PDF 格式，需要包含详细的实验过程与实验结果分析）
- 2. 完整项目代码（包含训练、测试及相关依赖文件）
- 3. 训练参数文件（包括配置文件、超参数设置等）
- 4. 训练完成的模型文件（用于结果复现与测试验证）

命名为 "学号_姓名_大作业" ，打包发送到邮箱：

```
robotics_2026@163.com
```

截止日期： 2026.7.12 23:59